EBU6304 Software Engineering Group Project

2025-26

Group number	40	
member 1	QM no: 231220873	Name: Haocheng Gao
member 2	QM no: 231221283	Name: Xinshuang Liu
member 3	QM no: 231221700	Name: Chuan Yu
member 4	QM no: 231221250	Name: Ziqi Gao
member 5	QM no: 231221412	Name: Xianggan Xu
member 6	QM no: 231221777	Name: Zhibo Zhang

First report

1. Requirements

1.1 Fact-finding Techniques

In this project, we used four main fact-finding techniques: background reading, interviewing, observation, and document analysis. These techniques helped us understand the current TA recruitment process, identify the needs of the main stakeholders, and define the scope of the initial system more clearly.

First, we carried out background reading on the project handout and the existing TA recruitment process used in the university, while also reviewing Boss Zhipin as a reference recruitment platform. This helped us clarify the overall scope of the system, the roles it needed to support, and how similar processes are presented in practice. From the handout and the current process, we understood that TA recruitment still relies heavily on forms and Excel files, and that the system should support not only applicants, but also module organisers and administrative staff involved in the recruitment workflow. Based on this review and our early fact-finding, we defined TA recruitment as a unified workflow covering both module-related support and other school activities, such as invigilation, rather than separating them into different systems, because they share the same core process of profile preparation, application, review, and workload management. At the same time, by comparing the current process with Boss Zhipin, we examined how vacancy information is displayed, how users browse and apply for positions, how feedback is provided after submission, and how the interface is organised. This helped us identify several ideas that could be adapted to our project, such as clearer information display, a more structured application flow, better visibility of progress, and useful design references for the prototype, particularly in page layout, information hierarchy, and navigation between key steps.

Second, we used a small-scale workflow observation to understand the application process from the applicant's perspective. We observed one postgraduate applicant completing a TA application form and then held a short follow-up discussion to understand what happens in the later stages of the process. Although this observation was limited to one case, it helped

us see how applicants interpret requirements, prepare materials, complete the form, and think about what happens after submission. The observation suggested two main issues: some application requirements were not sufficiently clear at the point of submission, and applicants had limited visibility of what would happen after they submitted their forms.

We also used document analysis to review the materials involved in the current recruitment process, including application forms, recruitment messages, and records showing how information is collected and processed. This helped us identify what data the system needs to capture, where repeated entry may occur, and which parts of the current workflow depend on manual handling. It also helped us understand the information needs of different roles more clearly.

Finally, we interviewed a current teaching assistant working on the course, as well as postgraduate applicants, to understand the TA recruitment process from multiple perspectives. The interview with the teaching assistant was particularly valuable because it not only reflected the experience of applying for a TA role, but also gave us a more practical understanding of how the process operates in a real course context. The discussion covered not only what information applicants need and where misunderstandings are likely to arise, but also process-related issues concerning module organisers and administrative staff, such as what information needs to be passed to administrators and what kinds of permissions module organisers may require. In addition, we showed her part of our prototype and discussed the interface structure, information presentation, and process flow, which provided direct input for later prototype refinement. The interviews with postgraduate applicants further showed that clear vacancy descriptions, explicit application requirements, a straightforward submission process, and visible status updates after submission are all important to users. Taken together, and considered alongside the handout and our process review, these interviews helped us develop a broader and more realistic understanding of the overall workflow.

By combining these techniques, we were able to identify both process-level problems and user-level needs. These findings provided a practical basis for the initial backlog and informed later decisions on prioritisation, iteration planning, and prototype design.

1.2 Iteration plan

The project is divided into four iterations. This arrangement is based, on the one hand, on the need to develop the system step by step according to the priority, dependency, and distribution of story points across the user stories; on the other hand, it is also intended to align with the staged assessment and version delivery schedule set out in the handout. Since the project requires the team to continue submitting working software at later stages, we tried to avoid placing too much work in Iteration 1 and instead spread the development tasks across four stages. This makes it possible to establish the core recruitment workflow first, and then continue to add management functions and usability improvements in later versions.

Iteration 1 is planned to establish the basic working workflow of the system. At this stage, the focus is on the minimum set of features needed to demonstrate the system's primary value. This includes login and role-based access, creating application materials, browsing vacancies, viewing job details, submitting applications, checking application status, creating positions, and viewing application lists. These stories were placed early because they form the basic recruitment chain for the three main roles in the system: applicants, module organisers, and administrators. This shared workflow is intended to cover both module-related support and other school activities such as invigilation. At the same time, the team intentionally avoided putting too many stories into this first iteration, so that the scope would remain realistic and the initial build could be completed on a stable basis.

Iteration 2 is planned to complete the main recruitment workflow and make the system more usable in practice. After the basic flow has been established, this iteration adds several important supporting functions, including updating application materials, withdrawing an application, preventing duplicate applications, viewing applicant details and resumes, reviewing applications and updating statuses, and allowing administrators to view the overall workload of teaching assistants. These stories were placed in the second iteration because they are closely tied to the core workflow, but depend on the existence of the basic application and position-management structure delivered earlier. By the end of this stage, the system is expected to support not only submission and viewing, but also the main review and allocation process.

Iteration 3 is planned to improve process management, transparency, and coordination. At this stage, the team will focus on features such as viewing application history, editing and closing positions, automatically sending review result notifications, exporting accepted applicant lists, setting application quantity limits, and viewing hiring and assignment status. These stories are important for making the system more complete and more practical in real use, but they are scheduled after the core workflow because the system can still function without them in the earlier versions. This stage therefore builds on the previous iterations by strengthening management-related functions and improving the clarity of the recruitment process for different stakeholders.

Iteration 4 is planned for enhancement features and further refinement. The stories assigned to this stage include filtering jobs by criteria, favouriting jobs for quick access, viewing recruitment statistics, and sorting applicants by match score. These functions can improve convenience, efficiency, and decision support, but they do not determine whether the core recruitment system can operate. For this reason, they were placed in the final iteration, after the main workflow and key management features had already been addressed. This also leaves room for adjustment if earlier iterations require more time or if feedback suggests that some functions need to be reprioritised.

Overall, this iteration arrangement gives the project a clearer structure and makes the development pace easier to manage. The first two iterations focus on completing the core recruitment workflow of the system, while the later two build on this by adding management-related functions and improving the user experience. With this distribution, the team can

complete a basic working version first and then gradually refine the system based on later feedback and the actual progress of development.

1.3 Prioritisation

In this project, we used the MoSCoW method to prioritise the user stories in the product backlog. When making these decisions, we did not simply look at whether a function was useful. We considered whether it was needed for the basic TA recruitment workflow, whether it would add clear value at the current stage of development, and whether postponing it would make later development or use more difficult. This helped us decide which stories should be addressed first and which could be scheduled later across the shared TA recruitment workflow.

The Must have category was used for stories that support the basic operation of the system. These are the functions needed to establish the main recruitment workflow for the different roles involved, such as browsing vacancies, updating materials, viewing job details, submitting applications, checking application status, posting positions, and reviewing applicants. These stories were given the highest priority because they form the foundation of the system and need to be in place before later functions can be added.

The Should have category was used for stories that are important for making the system more complete and practical in real use, but are not the first features needed in the earliest working version. These include functions such as withdrawing applications, preventing duplicate submissions, viewing applicant details more clearly, and supporting workload-related checks. These stories were still treated as important, but were placed after the Must have items because they depend on the core workflow already being available.

The Could have category was used for stories that mainly improve convenience, efficiency, or ease of use. Examples include filtering vacancies, favouriting jobs, improving information organisation, and other interface-related improvements. These functions can make the system easier to use, but they are not necessary for establishing the basic recruitment process, so they were given lower priority.

The Want to have category was used for ideas that have lower immediate importance at the current stage or are more suitable for later extension. These stories were not removed from the backlog, but were kept as optional items that could be considered if time and progress allow. This helped us keep the scope under control and focus first on the stories that were more closely related to the main workflow.

Overall, this prioritisation helped us focus first on the stories needed to establish the core recruitment process, and then gradually add supporting functions and user experience improvements. It also made the later iteration planning clearer, because the more necessary stories could be arranged earlier, while less urgent items could be left for later stages.

1.4 Estimation Methods

In this project, we used story points to estimate the size of the user stories in the product backlog. We chose this method because, at this stage of the project, it was more useful to compare the relative size and complexity of different stories than to give exact time estimates in hours. This made discussion easier within the team and also helped us plan the workload of each iteration more realistically.

When assigning story points, we used relative sizing rather than trying to predict precise development time. In practice, we mainly considered how much logic a story involved, how many steps were included in the workflow, whether it required validation or status changes, and how much uncertainty it might introduce during development. Based on this, simpler stories, such as basic information display or lightweight status updates, were given lower points, while stories involving core workflows, multiple steps, or more complex interaction were given higher points.

Story points were discussed and agreed by the team using relative comparison across the backlog rather than by assigning time estimates individually. In practice, we compared each story with other stories that were already better understood, so that the team could reach a shared judgement about whether a story was relatively small, medium, or large.

We used a Fibonacci-style scale, such as 1, 2, 3, 5, and 8, because this non-linear pattern reflects the fact that larger stories usually involve disproportionately more uncertainty and effort. In our backlog, stories worth 1–2 points were generally simple display or query functions, 3-point stories usually involved moderate logic or validation, and 5-point stories were typically core process functions involving several steps or state changes. When a story was estimated at 8 points, we treated it as a relatively large item that needed further attention. In some cases, this could suggest that the story might be split into smaller parts. However, if the story had relatively low priority and was not planned for early implementation, it could also remain as it was and be reconsidered later.

This method also supported later planning decisions. Because the team estimated stories in relative terms, it became easier to compare workload across the backlog, avoid placing too much work in a single iteration, and distribute tasks more reasonably across the four planned iterations. In this way, estimation was not used as a precise promise of development time, but as a practical tool for planning, discussion, and adjustment during the project.

2. Supporting material

2.1 Core Interview Questions

How does the current TA application process usually work?

What are the most inconvenient or unclear parts of the current process?

What information and materials do applicants usually need to submit when applying?

Does the current process involve repeated entry of the same information?

After submitting an application, are applicants usually able to know the later progress in time?

What key information does a module organiser need to provide when posting a vacancy?

What support does a module organiser need most when screening and selecting TAs?

What information does an administrator need to view and manage most?

Should the system support an application deadline, automatic closing of applications, or application withdrawal?

Which functions should be treated as core functions that must be completed first?

Which functions could be treated as later extensions?

In what ways does the current prototype still need improvement?

2.2 Selected Supporting Evidence

The materials included in this section are selected examples rather than a complete record of all discussions and evidence collected during the project. They are presented here to support the main findings discussed in the report. To avoid unnecessary repetition, only the most relevant screenshots, records, and prototype materials have been included.

A. Evidence on the Current Manual TA Application Process

The following figures provide representative examples of how the current TA application process operates in practice. They show that recruitment information was shared through informal channels, that applicants were required to complete external forms, and that the process relied heavily on manual submission and deadline-based handling. Only a small number of examples are shown here, as similar records have been omitted to avoid duplication.

Figure 1. Screenshots illustrating the current TA application process. Vacancy information is communicated through chat messages and applicants are asked to complete an external form, which reflects a fragmented and manual recruitment workflow.

B. Interview Evidence on Stakeholder Needs

The following records are selected examples from the team's interviews and discussions with applicants and related participants. They were used to understand the main problems in the current workflow, identify user needs, and discuss issues related to applicants, module organisers, and administrative staff. These examples are intended to support the findings summarised in the report rather than reproduce every conversation in full.

Figure 2. Discussion records from the team's fact-finding process, showing how current workflow problems and user needs were identified, including application information requirements, vacancy management, administrative functions, and process visibility.

C. Reference Design and Prototype Development Evidence

The following materials show how the team used competitive analysis and early prototype exploration during the design stage. Screenshots from Boss Zhipin were used as reference examples for vacancy presentation, interface layout, and information organisation, while the sketch and related discussion records show how these ideas were translated into the team's own prototype framework. Repetitive examples have been omitted, and only the most representative materials are included here.

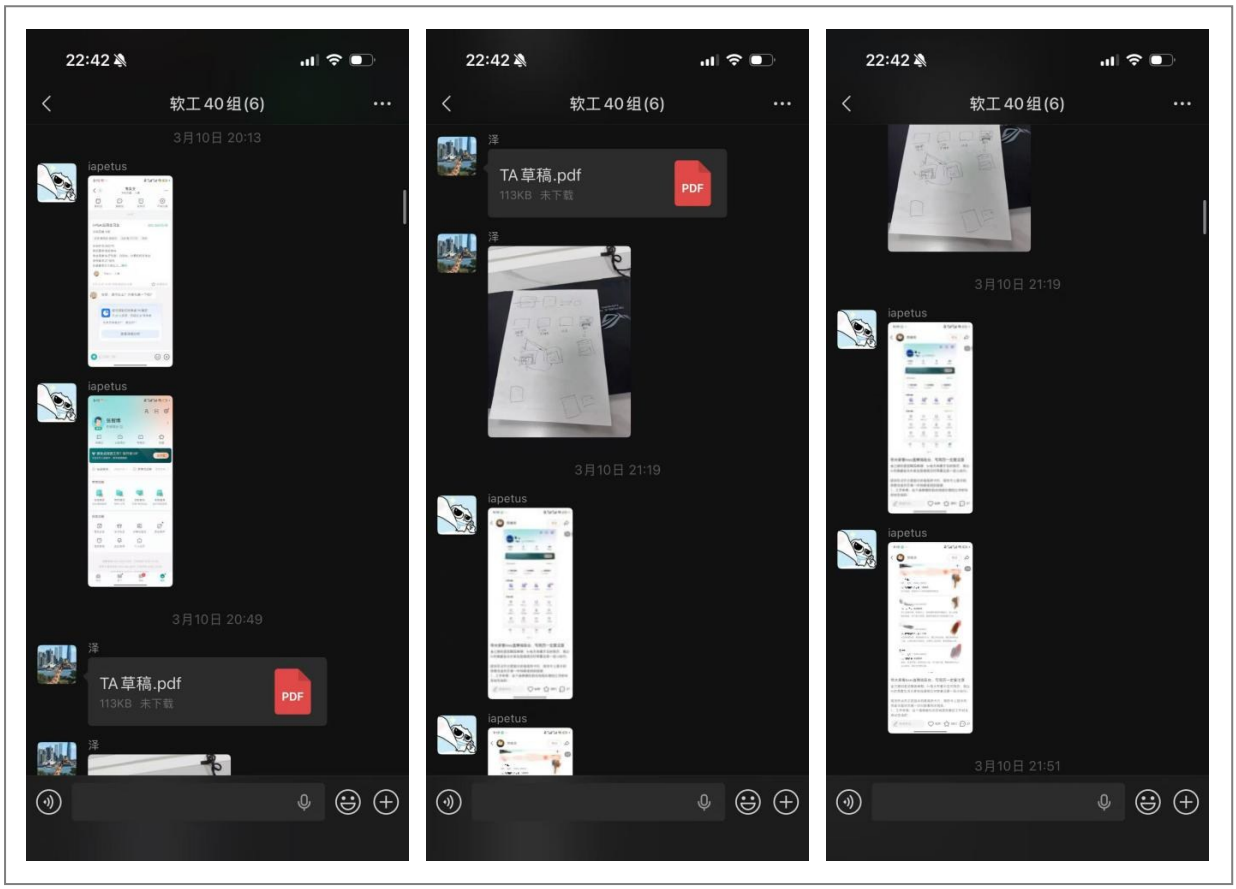

Figure 3. Materials from the competitive analysis and early prototype development stage. Boss Zhipin screenshots were used as reference examples for interface layout, vacancy presentation, and information organisation, while the draft PDF and hand-drawn sketch show how these ideas were translated into the team's own prototype structure.

D. Evidence Supporting Technical and Scope Decisions

The following record shows the team's discussion with the allocated teaching assistant on technical constraints and implementation options. It was used to clarify the acceptable development forms and storage limitations described in the handout, and to ensure that later design decisions remained consistent with the coursework requirements.

Figure 4. Teaching assistant feedback used to clarify technical constraints in the handout. The discussion confirmed the acceptable forms of implementation and helped ensure that the team's design decisions remained consistent with the coursework requirements.